

Birla Institute of Technology & Science, Pilani Hyderabad Campus**CS F372: Operating Systems (Assignment No: 3)****Assigned: 01/04/2026****Date of Submission: 17/04/2026****Total Marks: 12****Weightage: 6%****General Instructions:**

- This assignment can be done in groups of **maximum 4** students.
- All your programs must run on a **Ubuntu 24.X virtual machine on Oracle Virtual Box**.
- Recommended allocation: 4 cores, 50 GB virtual disk, 4GB RAM or more on VM.
- Submit all source files and executables along with a readme that has group details (Names and ID numbers of all members of the group.)
- Put all your deliverables in a zip/tar file and submit through course webpage (assignment section). One submission per group is sufficient.
- Every assignment will have a demo and viva, the schedule of which will be intimated later through a separate notice.

For any assignment related queries, you may contact I/C or the following teaching assistants:

- Arpit Kudre <f20230158@hyderabad.bits-pilani.ac.in>
- Parth Munjal Joshi <f20230127@hyderabad.bits-pilani.ac.in>

Q1. (4 Marks) The Sleeping Dentist:**Problem Overview:**

The dentist at the medical center of BITS Hyderabad attends to all patients during working hours. The clinic consists of a single examination room where the dentist examines patients individually. Outside this examination room, there are **5 waiting chairs** where patients might sit if the dentist is busy.

When there are no patients present in the clinic, the dentist sleeps. If some patient arrives and finds the dentist sleeping, they should wake the dentist up to receive a consultation. If a patient arrives and the dentist is busy, they should wait while sitting on one of the 5 chairs if any of the chairs are free. If all the chairs are occupied, the patient goes away and comes back at a later time.

Patients have limited patience and limited time. After waiting for **k seconds** on the chairs, they will give up and leave. After leaving, whether due to a full waiting room or a timeout, the patient returns after **y seconds**. In addition to this, if the patient does not get a consultation after **x visits**, they are left untreated.

Your task is to implement a simulation and generate a clinic report which tracks all these parameters with the help of POSIX threads, semaphores and mutexes. The dentist runs as one thread and each of the patients runs as a separate thread. The main thread is responsible for reading the input, spawning all the other threads, waiting for them to complete and printing the final report.

Input Format:

Your program must read from a file named input.txt. The first line contains four integers:

- **n** – Number of patients
- **k** – Maximum seconds a patient will wait on a chair before leaving
- **x** – Maximum number of visits a patient will try before giving up permanently
- **y** – Number of seconds a patient waits before returning to the clinic after timing out or being turned away due to full chairs

Each of the following n lines contains a single integer which represents the arrival time in seconds of that patient. Patients are numbered from 1 to n in the order they appear in the input file.

Sample **input.txt**:

```
10 2 2 3
0
0
0
0
0
0
0
0
0
1
2
```

Synchronization Requirements:

You should use these POSIX synchronization primitives:

- **Mutex Lock** – to protect the number of occupied waiting chairs.
- **Semaphore** – You will need 2 types of semaphores:
 - **dentist_sleep**: The dentist waits on this semaphore when no patients are present.
 - **patient_ready**: An array of semaphores where the waiting patient blocks on its semaphore (patient_ready[i]) until a dentist is ready to see them.

Behaviour Specification:**Dentist Thread:**

1. Wait on **dentist_sleep** semaphore when no patients are present.
2. When woken, scan the patients waiting and find the one with the lowest id.
3. Simulate a consultation by sleeping for 1s.
4. Check if there are any patients waiting after consultation.
5. If yes, repeat from step 2, else go back to sleep.
6. When there are multiple patients waiting, the dentist serves the patient with the **lowest patient number(or patient ID)**.
7. If there are no patients waiting, return to sleep.
8. The dentist thread exits once all patients have been helped or have exhausted their max number of visits.

Patient Thread:

1. Sleep until the arrival time.
2. Repeat until the patient is treated successfully or reaches the visit count
 - a. Acquire the mutex to check the number of chairs.
 - b. If chairs are full, release the mutex, and sleep for stipulated time before retrying.
 - c. If there is a free chair, sit and increase the waiting count before releasing the mutex.
 - d. If the patient is the only one sitting, signal the dentist.
 - e. Wait for at most **k seconds** before receiving a consultation or leaving.
 - f. If there is a timeout, leave by decrementing the chair count, and sleep.
 - g. If the dentist signals, receive the consultation, and then exit
 - h. Make sure to record all timeouts and turnaways.
3. After **x total** attempts if the patient still does not get a consultation, exit regardless.

(Hint: Make sure to handle the case where the dentist calls the patient at the same moment of the timeout.)

Output Format:

For each patient (1 to n), print one line in the following format:

```
Patient <id> - Status: TREATED/UNTREATED | Visits: <v> | Timeouts: <t>
| Turnaways: <u>
```

At the end, print:

```
Total treated: <count>
Total untreated: <count>
```

Definitions:

- **Visits:** total number of times the patient attempted to enter the clinic
- **Timeouts:** number of visits where the patient sat in a chair but left because *k seconds* elapsed with no consultation
- **Turnaways:** number of visits where the patient found all chairs full and left immediately

Sample output:

```
--- CLINIC REPORT ---
Patient 1- Status: TREATED | Visits: 1 | Timeouts: 0 | Turnaways: 0
Patient 2- Status: TREATED | Visits: 1 | Timeouts: 0 | Turnaways: 0
Patient 3- Status: TREATED | Visits: 2 | Timeouts: 1 | Turnaways: 0
Patient 4- Status: TREATED | Visits: 2 | Timeouts: 1 | Turnaways: 0
Patient 5- Status: UNTREATED | Visits: 2 | Timeouts: 2 | Turnaways: 0
Patient 6- Status: UNTREATED | Visits: 2 | Timeouts: 2 | Turnaways: 0
Patient 7- Status: TREATED | Visits: 2 | Timeouts: 0 | Turnaways: 1
Patient 8- Status: TREATED | Visits: 2 | Timeouts: 0 | Turnaways: 1
Patient 9- Status: TREATED | Visits: 1 | Timeouts: 0 | Turnaways: 0
Patient 10- Status: TREATED | Visits: 2 | Timeouts: 1 | Turnaways: 0

Total treated: 8
Total untreated: 2
```

NOTE: Logical time units may be simulated using smaller real-time delays (e.g., milliseconds) for faster execution.

Q2. (4 Marks) The Martian Rover Maintenance Hub:**Problem Overview:**

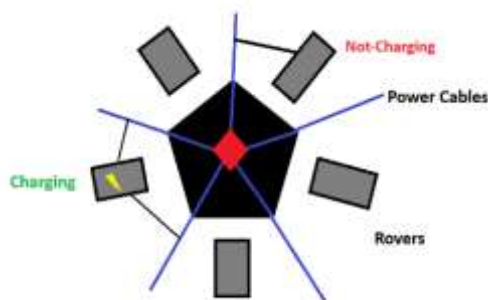
You are programming the synchronization software for a circular solar-charging hub on Mars. There are five autonomous rovers (numbered 0 through 4), each simulated as a separate thread. The hub has five heavy-duty charging cables situated between the rovers. Because of the extreme power requirements, a rover must plug in two adjacent cables (left and right) to successfully charge.

Rover Behaviour:

Each rover thread alternates between two states:

1. **Exploring:** The rover is out in the field. To simulate this, the thread must sleep for a random period between one and five seconds.
2. **Charging:** The rover is at the hub using cables. To simulate this, the thread must sleep for a random period between one and three seconds.

An illustration of the scenario is given below:

**Technical Constraints:**

- You must strictly use `pthread_mutex_t` and `pthread_cond_t` to manage state.
- You must ensure that your solution is free of deadlocks.
- You must print when a rover's state changes.

Battery Based Priority:

Some of you might have realised that this problem is quite like the classic dining philosopher's problem. Unlike philosophers though, who can wait indefinitely, rovers have batteries. Each rover thread must track its own battery level. If a rover's battery drops below a critical threshold (20%), it enters an *Emergency State*. The synchronization mechanism must ensure that an emergency rover gets priority access to the cables as soon as they become available, preventing starvation. A rover's battery depletes by 10% per second while it is exploring. A rover's battery depletes by 5% per second while it is waiting to charge. All rovers start with 50% charged batteries. If a rover's battery reaches 0%, it becomes permanently stranded. The program must log a critical failure message and terminate that rover's thread. Rovers are charged by 15% for every second they spend charging.

Sample Output:

To verify your solution, the program must print a log of events to the console. Each line should include the Rover ID and its current battery level:

```
Rover 0 is waiting for cables (Battery: 10%)
Rover 0 ENTERING EMERGENCY STATE (Battery: 10%)
Rover 0 is charging (Battery: 10%)
Rover 3 finished charging (Battery: 85%)
Rover 3 is exploring (Battery: 85%)
Rover 2 is charging (Battery: 15%)
Rover 1 STRANDED: Battery depleted to 0% during exploration.
```

(Hint: Because C does not have a built-in monitor construct, you must implement the monitor logic using Pthreads Mutexes and Condition Variables.)

Q3. (4 Marks) The Second Readers-Writers Problem (Writer Preference):

Problem Overview: The Readers-Writers problem is a classic synchronization challenge involving a shared data area (such as a database or file) accessed by multiple processes. These processes are categorized as:

- Readers: Processes that only need to read data and never modify it.
- Writers: Processes that can both read and write (update) the data.

The Constraint: In the Second Readers-Writers Problem, the synchronization must implement Writer Preference. This means that once a writer becomes ready to write, no new readers are allowed to start reading. The goal is to prevent writers from being delayed indefinitely by a continuous stream of readers.

Functional Requirements: To ensure data integrity and proper synchronization, your solution must satisfy the following conditions:

1. **Mutual Exclusion:** Only one writer may access the shared data at any given time.
2. **Concurrent Reading:** Multiple readers may access the shared data simultaneously, provided no writer is currently writing or waiting to write.
3. **Writer Priority:** If a writer is waiting to access the data, any new readers that arrive must wait until the writer has finished its task.
4. **No Starvation (Writers):** A writer should not be kept waiting longer than necessary; once a writer expresses intent to write, it should gain access as soon as the current active readers finish.

Technical Constraints & Simulation Parameters: To properly simulate this environment and standardise submissions, you must adhere to the following setup:

- **Synchronization Primitives:** You must strictly use POSIX Semaphores (<semaphore.h>) to manage access, state, and priority blocking. Do not use mutexes or condition variables for this specific question.
- **Thread Counts:** The main function must spawn exactly 4 reader threads and 2 writer threads.
- **Shared Data:** Maintain a single shared integer resource initialized to 0.
- **Writer Behaviour:** When a writer enters the critical section, it must increment the shared resource by 5. To simulate the writing process, the thread must sleep for a random duration up to 1.5 seconds. After finishing, it should sleep for a random duration up to 5 seconds before attempting to write again.
- **Reader Behaviour:** To simulate reading, the thread must sleep for a random duration up to 1 second. After reading, it should sleep for a random duration up to 3 seconds before attempting to read again.

Logging Requirements: Your program must print the exact state of the threads when they access the resource. Ensure your output matches the following format:

- Reader Output: [Reader <id>] Reading resource: <shared_resource_value> (Active Readers: <current_read_count>),
- Writer Output: [Writer <id>] Writing to resource: <shared_resource_value>

Sample Output:

```
[Reader 1] Reading resource: 0 (Active Readers: 1)
[Reader 4] Reading resource: 0 (Active Readers: 3)
[Reader 2] Reading resource: 0 (Active Readers: 2)
[Writer 1] Writing to resource: 5
[Writer 2] Writing to resource: 10
[Reader 3] Reading resource: 10 (Active Readers: 1)
```